

01

HWPX — 한글 파일의 잠긴 뚜껑을 여는 법

확장자 하나만 바꾸면, 한글 프로그램 없이도 그 안의 글자와 서식이 전부 드러납니다. 그리고 그 순간부터 자동화가 시작됩니다.

1

마주친 장면과 정의

누군가 **공문_최종.hwp** 파일을 보내왔습니다. 그런데 이 컴퓨터에는 한글 프로그램이 깔려 있지 않습니다. 더블클릭하면 "이 파일을 열 앱을 선택하세요"라는 창만 뜹니다. 대부분은 여기서 포기하고, 한글을 설치할 방법을 찾거나 상대방에게 PDF로 다시 보내 달라고 부탁드립니다.

그럴 필요가 없습니다. 이 파일은 사실 **압축파일**입니다. 겉으로는 한글 문서처럼 보이지만, 속을 열어 보면 여러 개의 작은 파일이 폴더 구조로 담겨 있는 상자입니다. 상자를 여는 열쇠는 특별한 프로그램이 아니라, 이미 컴퓨터 안에 있는 압축 해제 기능입니다.

확장자 신상명세

확장자	.hwp (에이치더블유피엑스)
정식 이름	한글 문서 개방형 포맷 (OWPML 기반)
정체	ZIP 압축파일 + 그 안의 XML 여러 장
만든 곳	한글과컴퓨터 · 2010년대 표준화
왜 생겼나	옛 .hwp는 속을 뜯기 어려운 이진 파일. 공공기관 문서를 누구나 열 수 있게 만들려고 개방형으로 다시 설계
친척	.docx .xlsx .pptx .epub — 모두 같은 방식(ZIP+XML)

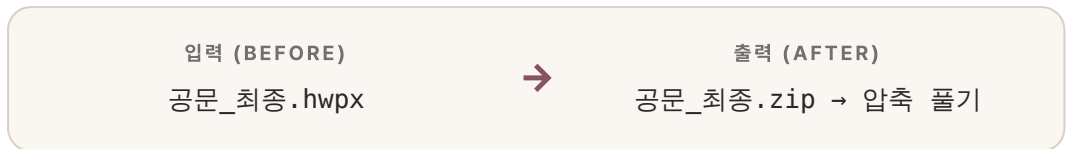
§ 한 문장으로

HWPX는 **여러 개의 XML 파일을 ZIP으로 묶은 문서 상자**입니다. XML은 글자에 이름표를 붙여 정리해 둔 텍스트 형식이고, ZIP은 그것들을 하나로 압축해 담은 봉투입니다. 그래서 봉투를 풀면 이름표가 붙은 글자들이 그대로 나옵니다.

XML(Extensible Markup Language)은 **<제목>예산안</제목>** 처럼 내용마다 여는 표와 닫는 표로 감싸 두는 텍스트 형식입니다. 사람이 읽어도 뜻을 짐작할 수 있고, 프로그램은 표를 보고 정확히 그 자리를 찾아냅니다.

§ 바꾸기 전과 후

가장 먼저 눈으로 확인할 변화는 이렇습니다. 확장자 이름 세 글자를 바꾸는 것으로, 열리지 않던 파일이 폴더처럼 열립니다.



압축을 풀면 **mimetype**, **Contents** 폴더, **settings.xml** 같은 파일이 줄줄이 나타납니다. 이 목록을 처음 본 사람은 대부분 같은 반응을 보입니다. "한글 파일 안이 이렇게 생겼다고?"

2 상자를 해부한다

상자를 열었으니 안을 들여다봅니다. HWPX 안의 파일들은 아무렇게나 담겨 있지 않고, 역할이 정해진 방마다 나뉘어 있습니다. 아래 그림은 그 방 배치도입니다.



그림 1. HWPX를 압축 해제하면 나오는 방 배치. 글자를 바꾸고 싶다면 목적지는 단 하나, Contents/section0.xml입니다.

핵심은 두 개의 방입니다. **header.xml**은 "글자를 어떻게 보이게 할지"의 규칙을 담고, **section0.xml**은 "실제로 무슨 글자가 적혀 있는지"를 담습니다. 우리가 내용을 읽거나 바꾸고 싶을 때 찾아갈 곳은 언제나 **section0.xml** 하나입니다.

표 1. HWPX 상자 안의 주요 파일과 역할

파일 / 폴더	맡은 일	바뀌어도 되나
mimetype	"이 파일은 HWPX"라는 신분증	건드리지 않음
Contents/header.xml	글꼴·문단·스타일 정의	서식 바꿀 때만
Contents/section0.xml	본문 글자 (<hp:t> 안)	여기를 바꾼다
settings.xml	커서 위치 등 편집 상태	보통 그대로
BinData/	넣어 둔 이미지 파일	그림 교체 시

● 손으로 해보기 · 마우스만으로

프로그램을 새로 깔지 않습니다. 윈도우 탐색기 또는 맥 파인더만으로 따라 할 수 있습니다.

- 1 확장자를 보이게 합니다.** 윈도우 탐색기 상단 '보기'에서 '파일 확장명'에 체크합니다. (맥은 파일 정보에서 이름 끝을 확인)
- 2 사본을 하나 만듭니다.** 원본이 상하지 않도록 **공문_최종.hwp** 를 복사해 둡니다.
- 3 확장자를 바꿉니다.** 사본 이름의 **.hwp** 를 **.zip** 으로 고칩니다. "정말 바꾸겠냐"는 경고에 예를 누릅니다.
- 4 압축을 풉니다.** 우클릭 → '압축 풀기'. **Contents** 폴더가 보입니다.

5

속을 확인합니다. `Contents/section0.xml` 을 메모장이나 브라우저로 열어 봅니다. 낫선 태그 사이에 문서의 진짜 글자가 보입니다.

되돌리기: 방금 만든 사본이니 지워도 원본은 그대로입니다. XML을 고친 뒤 다시 `.hwp` 로 쓰는 방법은 이 장 뒷부분에서 다룹니다.

3

이 걸로 무엇을 자동화하나

여기까지는 "열어 보기"였습니다. 진짜 가치는 다음 질문에서 나옵니다. **이 구조를 알면 어떤 일을 자동으로 시킬 수 있는가.** 문서가 상자이고 글자가 특정 방에 정해진 이름표로 들어 있다면, 그 이름표를 찾아 갈아 끼우는 일은 사람이 아니라 도구가 할 수 있습니다.

문서가 구조를 가진 순간, 반복 작업은 노동이 아니라 명령이 됩니다.

§ 대표 시나리오 — 공문 100장을 1초에

받는 사람 이름과 날짜만 다른 안내 공문을 100명에게 보내야 한다고 합시다. 사람이 한글을 열어 이름을 바꾸고 다른 이름으로 저장하기를 100번 반복하면 한나절이 걸립니다. 구조를 알면 과정은 이렇게 바뀝니다.

표 2. 사람이 하던 일 vs 구조를 이용한 자동화

단계	사람이 손으로	구조를 이용하면
원본 준비	한글로 서식 작성	한글로 서식 작성 (여기까지는 동일)
이름 바꾸기	100번 타이핑	이름 목록(엑셀/CSV)에서 자동 대입
바꾸는 위치	눈으로 찾아 수정	section0.xml 의 <hp:t> 만 교체
다시 저장	100번 '다른 이름으로'	ZIP으로 다시 묶어 100개 생성
걸린 시간	반나절	수 초

원리는 앞에서 본 그대로입니다. 상자를 열고(압축 해제), 본문 방의 이름표만 바꾸고 (<hp:t> 교체), 다시 봉투로 묶습니다(재압축). 이 세 동작을 100번 자동으로 돌리는 것뿐입니다.

● 지켜야 하는 규칙 · 어기면 문서가 깨진다

자동으로 상자를 다시 묶을 때는 몇 가지 약속이 있습니다. 이 약속을 어기면 한글이 "손상된 파일"이라며 열기를 거부합니다. 규칙을 아는 것 자체가 이 장의 실전 지식입니다.

규칙	여기면 벌어지는 일
<code>mimetype</code> 을 맨 처음에, 압축하지 않고(STORED) 넣는다	한글이 파일 자체를 거부
원본 파일 순서를 그대로 유지한다	열리다가 깨짐
글자는 문단의 첫 <code><hp:t></code> 에만 넣는다	같은 글자가 두 번 찍힘
표는 원본 행·열 범위 안에서만 채운다	행을 늘리면 표 구조 파손
<code>section0.xml</code> 이 쓰는 스타일 번호는 <code>header.xml</code> 에 있어야 한다	조용히 서식이 사라짐

이 방식에는 이름이 있습니다. 원본 상자의 구조는 손대지 않고 글자만 갈아 끼운다고 해서 **템플릿 보존 방식**이라고 부릅니다. 구조를 안 건드리니 한글에서 100% 정상으로 열리고, 문서를 밖으로 보내지 않으니 개인정보가 새어 나가지 않습니다.

밖으로 보내지 않는다는 점이 실무에서 특히 중요합니다. 이름·주민등록번호가 든 공문 100장을 인터넷 변환 사이트에 올리지 않고, 내 컴퓨터(또는 브라우저) 안에서만 처리하면 그 자체로 개인정보 유출 위험이 사라집니다.

4

AI에게 시키는 법과 주의점

여기서 중요한 전환이 있습니다. 앞의 규칙들을 **사람이 외워서 직접 코딩할 필요가 없습니다**. 규칙을 아는 것은 코드를 쓰기 위해서가 아니라, **AI에게 정확히 지시하기 위해서**입니다. 무엇을, 어디를, 어떤 규칙 아래 바꿔야 하는지 알면, 실제 작업은 AI 코딩 도구에게 맡길 수 있습니다.

§ 먼저: 그냥 열어 보고 내용만 뽑기

코드를 만들 것도 없이, HWPX 파일을 AI에게 그대로 건네고 물어보는 것부터 시작합니다. 대화형 AI는 압축을 풀고 `section0.xml`의 글자를 읽어 줍니다.

> _ 대화형 AI에게 · 붙여넣기

이 .hwpx 파일은 ZIP+XML 구조야.
압축을 풀고 Contents/section0.xml 안의
<hp:t> 태그에 든 본문 텍스트만 순서대로 뽑아서
깨끗한 글로 정리해 줘. 표는 표 모양 그대로 옮겨 줘.

§ 다음: 공문 100장 자동 생성 도구 만들기

이제 반복 작업을 처리할 작은 도구를 AI 코딩 도구(예: Claude, Cursor)에게 만들라고 시킵니다. 이때 앞에서 배운 규칙을 지시문에 그대로 적어 주는 것이 성공의 핵심입니다.

> _ AI 코딩 도구에게 · 지시문

브라우저에서만 도는 HTML 파일 하나를 만들어 줘. 서버는 쓰지 마.

할 일: 템플릿 .hwpx 하나와 이름·날짜가 든 CSV를 올리면,
각 줄마다 공문 한 장을 만들어 ZIP으로 한꺼번에 내려받게 해 줘.

반드시 지킬 규칙:

- JSZip으로 hwpx를 풀고 다시 묶어.
- 본문은 section0.xml의 <hp:t>만 바꿔. 나머지 파일·순서는 그대로 뒀.
- mimetype은 맨 앞에, 무압축(STORED)으로 넣어.
- 완성 파일 확장자는 다시 .hwpx로.

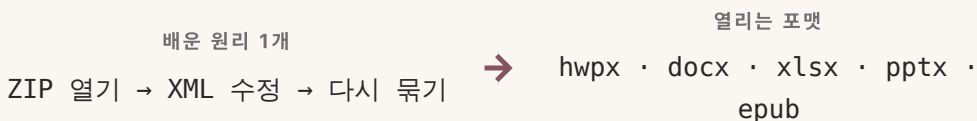
이 지시문이 잘 작동하는 이유는 분명합니다. "알아서 잘 만들어 줘"가 아니라, 목적지 (`section0.xml`)와 바꿀 대상(`<hp:t>`)과 깨지지 않을 규칙(`mimetype` 순서)을 꼭 집어 줬기 때문입니다. 확장자의 구조를 이해한 사람만 쓸 수 있는 지시문입니다.

§ 이 과정이 보장하지 않는 것

정직하게 한계를 적어 둡니다. 첫째, 이 방법은 **글자와 간단한 표 바꾸기**에 강하지, 복잡한 도형이나 수식이 얹힌 문서를 자유롭게 재편집하는 데는 적합하지 않습니다. 둘째, 표의 **행 수를 늘리는 일**은 구조를 건드리므로 템플릿 보존 방식의 범위를 벗어납니다. 셋째, AI가 만든 도구라도 **첫 결과물은 반드시 한글에서 직접 열어 확인**해야 합니다. 열리는 것과 올바른 것은 다릅니다.

§ 하나를 알면 열리는 가족들

가장 큰 수확은 따로 있습니다. 이 원리는 HWPX만의 것이 아닙니다. **.docx** (워드), **.xlsx** (엑셀), **.pptx** (파워포인트), **.epub** (전자책)도 전부 **ZIP+XML** 구조입니다. 상자를 열고, 글자가 든 XML을 찾고, 바꾸고, 다시 묶는다는 한 가지 원리를 익히면, 오피스 문서 포맷 전체가 같은 방식으로 열립니다.



이 장의 3줄 요약

1. HWPX는 특별한 파일이 아니라 **XML**을 **ZIP**으로 묶은 상자다. 확장자를 **.zip**으로 바꾸면 누구나 열 수 있다.
2. 내용은 **section0.xml**의 **<hp:t>**에 있고, 이곳만 갈아 끼우는 **템플릿 보존 방식**으로 공문 100장을 몇 초에 만든다.
3. 규칙을 아는 이유는 코딩이 아니라 **AI에게 정확히 지시**하기 위해서다. 같은 원리가 docx·xlsx·pptx·epub에도 통한다.

다음 장 질문 → HWPX 안에서 우리가 열어 본 그 <태그>들의 정체가 바로 XML이었습니다. 그렇다면 XML은 대체 어디서 와서, 브라우저는 왜 태어날 때부터 그것을 읽을 줄 아는 걸까요? — **02. XML** — 브라우저에 숨어 있던 만능 번역기